

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

Звіт про результати за темою наукового дослідження

**КРИПТОАНАЛІЗ СТАНДАРТУ
ШИФРУВАННЯ O'Z DST 1105:2009**

Виконала студентка
групи ФІ-23
Тимошенко Марія Андріївна

Науковий керівник:
Кандидат наук
Яковлев Сергій Володимирович

ВСТУП

Сучасні реалії інформаційної безпеки вимагають необхідність глибокого дослідження та тестування національних і міжнародних криптографічних стандартів. Надійність і безпечність використання будь-якої криптосистеми залежить не лише від математичної стійкості її моделі, але й від якості програмних реалізацій (імплементації) на кінцевих пристроях. У цьому контексті державний стандарт Узбекистану O'z DSt 1105:2009 становить значний науковий інтерес для світової криптографічної спільноти через свою нетипову, особливо для європейського ринку, математичну архітектуру. На відміну від більшості сучасних блокових шифрів, побудованих на базі класичних полів Галуа $GF(2^n)$, алгоритм O'z DSt 1105:2009 оперує в кільці лишків за модулем 256 ($\mathbb{Z}_{256}$) та базується на спеціальній алгебрі так званих діаматриць. Використання таких алгебраїчних структур вимагає окремого незалежного дослідження стійкості шифру до сучасних методів криптоаналізу.

Проблематика дослідження. Аналіз відкритих міжнародних наукових джерел свідчить про глибокий дефіцит досліджень архітектури O'z DSt 1105:2009. Ба більше, у вільному доступі повністю відсутня верифікована відкрита програмна реалізація цього шифру. Така ситуація призводить до виникнення концепту «безпеки через маловідомість» (Security through obscurity), що суперечить фундаментальному криптографічному принципу Керкгоффса.

В рамках даного етапу роботи було програмно реалізовано алгоритму шифрування даних (АШД) і проаналізовано його технічний опис, регламентований державним стандартом Узбекистану O'z DSt 1105:2009. Головною метою цієї частини роботи стало створення математично точної, оптимізованої програмної моделі симетричного блокового шифру, здатної комплексно функціонувати за приємний час в сучасних програмних середовищах. Окремим, критично важливим завданням став аудит

офіційної технічної специфікації стандарту, ідентифікація в ній³ математичних колізій та розробка інженерних рішень для їх усунення на етапі написання програмного коду. Варто окремо підкреслити, що на даному етапі дослідження назви функцій і внутрішніх примітивів у програмному коді не адаптувалися під термінологію європейського ринку, а були збережені в оригіналі (наприклад, *Aralash*, *BaytAlmash*, *Sur*, *ShaklSeansKalit*). Таке рішення дозволяє уникнути понятійних викривлень та забезпечує прозору покрокову верифікацію відповідності моделі до офіційної документації стандарту.

1 ОБҐРУНТУВАННЯ ВИБОРУ ІНСТРУМЕНТАРІЮ ТА АРХІТЕКТУРНЕ ПРОЄКТУВАННЯ

Вибір інструментарію для проєктування архітектури криптографічного програмного забезпечення вимагає особливого підходу і зваження багатьох факторів. Мова програмування та парадигма розробки безпосередньо впливають на безпеку роботи з пам'яттю, швидкість виконання обчислень та відсутність вразливостей на низькому рівні.

1.1 Обґрунтування вибору мови програмування

Для програмної реалізації шифру було обрано високорівневу мову програмування Python (версії 3.10+). Цей вибір обґрунтовується такими фундаментальними перевагами:

- **Нативна підтримка довгої арифметики (Arbitrary-precision arithmetic):** Згідно зі стандартом O'z DSt 1105:2009, алгоритм використовує надвеликі цілочисельні послідовності. Зокрема, розклад ключів базується на формуванні 672-бітового сеансово-етапного ключа (K_{se}). Тоді як мови програмування на кшталт C або Java мають жорсткі обмеження розмірності (64 або 128 бітів) і вимагають використання складних сторонніх бібліотек (наприклад, GMP), Python обробляє цілі числа довільної точності «з коробки», що прибирає ризики переповнення пам'яті (Integer Overflow) та гарантує математичну точність побітових зсувів.

- **Безпека управління пам'яттю (Memory Safety):** Криптографічні модулі часто можуть бути вразливими до виходу за межі буфера (Buffer Overflow). Вбудований у Python автоматичний збирач сміття (Garbage Collector) та строга типізація посилянь унеможливають прямий несанкціонований доступ до комірок пам'яті.

– **Високий рівень абстракції:** Синтаксис мови дозволяє подавати складні матричні перетворення у вигляді, що максимально наближений до математичних формул зі специфікації, що значно спрощує сприйняття коду, подальший його аудит та верифікацію.

1.2 Проєктування гібридної архітектури моделі

Для розробки було обрано мультипарадигмальний підхід, що поєднує об'єктно-орієнтоване програмування (ООП) та концепції функціонального програмування.

– **Функціональне ядро (Pure Functions):** Всі математичні перетворення, такі як діаматричне множення (`_diamatrix`), циклічні зсуви (`sur_enc`, `sur_dec`) та побайтові заміни (`byt_almash`), винесені за межі класу у вигляді ізольованих чистих функцій. Вони є детермінованими (однаковий вхід завжди дає однаковий вихід) і не змінюють глобального стану системи. Це дозволить зручно покрити кожен математичний крок алгоритму незалежними модульними тестами (Unit Tests).

– **Фасад управління станом (ООП):** Клас ASD виконує роль високомаршрутизованого контролера. Його завданням є одноразово виконати «важкі» підготовчі обчислення під час ініціалізації екземпляра (наприклад, генерацію раундових ключів, попереднє обчислення матриць для розшифрування та генерацію масивів S-Box). Після цього клас інкапсулює ці дані у прихованих атрибутах і надає зручний публічний інтерфейс (API) для шифрування масивів даних.

Висновки до розділу 1

Використання мови Python у поєднанні з гібридною програмною архітектурою дозволило створити надійну, математично точну та стійку до низькорівневих помилок модель. Розподіл задач між ізольованими

математичними функціями та класом-контролером значно підвищив рівень тестованості програмного забезпечення.

2 ТЕХНІЧНА РЕАЛІЗАЦІЯ БАЗОВИХ КРИПТОГРАФІЧНИХ ПЕРЕТВОРЕНЬ ТА РЕЖИМІВ РОБОТИ

Шифрування за стандартом O'z DSt 1105:2009 є ітеративною процедурою, що складається з 8 повних раундів. У даному розділі описано внутрішню механіку та особливості програмної реалізації кожного з етапів перетворення даних та алгоритмічні підходи для забезпечення роботи алгоритму в різних режимах.

2.1 Внутрішнє представлення даних (Матриця Holat)

Алгоритм працює з блоками даних фіксованої довжини — 256 бітів (32 байти). Для ефективного програмного використання лінійний потік байтів трансформується у двовимірну матрицю стану Holat. У програмній моделі вона реалізована як список списків цілих чисел розмірністю 8x4 (8 рядків по 4 стовпці).

Таке представлення матриці оптимізує процеси перемішування: доступ до будь-якого байта здійснюється за константний час $\mathcal{O}(1)$ за індексами $[row][col]$, що стає критично важливим для швидкодії на етапах циклічних зсувів та матричного множення.

2.2 Програмна імплементація примітивів дифузії та конфузії

Основою алгоритму є чотири послідовні криптографічні примітиви, які програмно реалізовано наступним чином:

- **Накладання ключа (Qo'shBosqichKalit):** Виконується операція побітового додавання за модулем 2 (XOR) між поточним станом матриці Holat та відповідним етапним ключем.
- **Діаматричне перемішування (Aralash):** Забезпечує глибоку

дифузію (розсіювання статистики). Матриця 8×4 ділиться на дві половини (4×4). Над кожною виконується діаматричне множення $\otimes_2$ за модулем 256. Для реалізації оберненої функції (при розшифруванні) в програмі імплементовано алгоритм Гаусса-Жордана, який знаходить обернену матрицю 16×16 у скінченному полі $GF(2^8)$.

– **Циклічні зсуви (Sur)**: Реалізовано двовимірне зсув елементів. Стовпці зсуваються циклічно вниз на величину $(j + 1) \pmod{8}$, а рядки — вправо на $(i + 1) \pmod{4}$. Математичне використання оператора залишку від ділення гарантує відсутність помилок виходу за межі масиву (Index Out of Bounds).

– **Нелінійна заміна (BaytAlmash)**: Забезпечує конфузію. Особливістю стандарту є динамічна зміна масивів (S-Box) B_{1A} та B_{2A} , які генеруються на основі сеансового ключа з використанням функції піднесення до степеня $x^d \pmod{257} \pmod{256}$. У програмному коді ці масиви генеруються на етапі ініціалізації (Pre-computation), що дозволяє під час шифрування виконувати миттєву заміну за індексом таблиці (Lookup Table).

2.3 Алгоритм вирівнювання (Padding) та режими шифрування

Оскільки реальні дані рідко бувають кратними 32 байтам, у модель було інтегровано стандартну схему вирівнювання PKCS#7. Функція доповнює останній блок n байтами, кожен з яких має значення n .

Для забезпечення практичної цінності моделі згідно зі стандартом реалізовано два класичні режими:

– **ECB (Electronic Codebook)**: Базовий режим, у якому кожен блок шифрується незалежно. Використовується переважно для тестування цілісності математичного ядра.

– **CBC (Cipher Block Chaining)**: Режим зчеплення блоків, що

забезпечує семантичну безпеку. Перед шифруванням поточного блоку над ним виконується операція XOR з попереднім шифротекстом (або Вектором Ініціалізації IV , для першого блоку).

Висновки до розділу 2

У програмній реалізації внутрішніх перетворень було суворо дотримано математичного апарату стандарту. Інтеграція динамічного розрахунку обернених матриць методом Гаусса-Жордана, впровадження стандарту доповнення $PKCS\#7$ та режиму CBC дозволили створити повноцінний інструмент шифрування інформації.

3 АУДИТ СПЕЦИФІКАЦІЇ СТАНДАРТУ ТА АЛГОРИТМІЧНЕ УСУНЕННЯ КОЛІЗІЙ

Під час програмування математичних формул з офіційного документа Державного стандарту Узбекистану O'z DSt 1105:2009 було виявлено ряд критичних неточностей та логічних помилок, які перетворювали алгоритм на непрацездатний. Даний розділ описує аналітичну роботу з виявлення цих колізій та обґрунтовує інженерні рішення щодо їх виправлення.

3.1 Усування колізії індексації діавизначника (Singular Matrix Problem)

Для коректної роботи перетворення Aralash діаматриці спеціальної структури K_1 та K_2 повинні бути строго оборотними. Стандарт описує процедуру примусового коригування парності елементів (п. 6.3.2.1) для забезпечення оборотності.

Суть проблеми: В офіційному тексті формула для обчислення одного з множників містить явну друкарську помилку:

$$k_{ss}[6] + k_{ss}[2] + k_{ss}[3] + k_{ss}[9] + k_{ss}[7] \pmod{2} = 0$$

Аналіз та рішення: Математичний аналіз структури діаматриці (п. 5.1.3 специфікації) показує, що четвертий множник діавизначника формується як сума $d_7 + d_2 + d_8 + d_9 + d_4$. Згідно з таблицею мапінгу елементів у офіційному документі, параметр d_9 відображається на байт $kss[4]$, а не на $kss[9]$.

У реалізованій програмній моделі (*_ensure_inv*) ця помилка специфікації ігнорується, і використовується прив'язка до математично правильних індексів $[2, 3, 4, 7]$. Завдяки цьому виправленню генератор сеансових ключів завжди створює матриці з відмінним від нуля визначником (Non-singular matrix).

3.2 Вирішення алгоритмічної помилки розкладу ключів розшифрування

Суть проблеми: У п. 6.3.5 стандарту наведено формулу початкового зсуву 672-бітового ключа K_{se} для процесу розшифрування: зміщення на $672 - (e \times 83) \pmod{672}$ бітів вліво. Експериментальна перевірка з використанням контрольних векторів (Додаток А) довела, що ця формула порушує синхронізацію: згенеровані таким чином ключі не є точним реверсом ключів шифрування, що призводить до незворотної втрати відкритого тексту.

Аналіз та архітектурне рішення: Оскільки специфікація розкладу ключів для дешифрування містить логічну помилку, було прийнято рішення від неї відмовитися на користь фундаментальної аксіоми симетричної криптографії.

У реалізованій моделі функція `build_epoch_keys` проводить повний процес шифрування, заздалегідь генеруючи масив з дев'яти коректних етапних ключів у прямій послідовності. Під час розшифрування алгоритм просто зчитує ці вже готові ключі у зворотному порядку (від останнього раунду до нульового). Це рішення обходить вади документації.

Висновки до розділу 3

Проведений технічний аудит специфікації виявив слабкі місця в текстовому описі стандарту. Виявлення друкарських помилок та не точностей опису індексації діавизначника та алгоритмічної колізії розкладу ключів (Key Schedule) є важливим науково-практичним результатом роботи. Застосовані інженерні обхідні шляхи забезпечили повну життєздатність розробленої моделі для подальшого її тестування на стійкість.

ВИСНОВКИ

В результаті виконання даного етапу роботи було успішно спроектовано та реалізовано справну повноцінну програмну модель національного стандарту криптографічного захисту інформації Узбекистану O'z DSt 1105:2009.

Ключові досягнення:

- **Ефективна архітектура:** Обрано мову Python та застосовано гібридну архітектуру (ООП + чисті функції), що дозволило вирішити проблему роботи з надвеликими числами та забезпечити високий рівень безпеки внутрішніх перетворень та тестованості коду.

- **Точна математична реалізація:** Програмно імплементовано всі складні матричні перетворення без використання сторонніх криптографічних бібліотек.

- **Практична адаптація:** Модель оснащена механізмом вирівнювання блоків (Padding) *PKCS#7* та підтримує класичні режими шифрування (*ECB*, *CBC*), що робить її придатною для інтеграції в реальні програмні продукти.

- **Усунення колізій стандарту:** Проведено глибокий критичний аналіз офіційної специфікації. Завдяки ідентифікації та виправленню помилок (в індексації матриць та логіці зсуву ключів), вдалося перетворити непрацездатний за описом алгоритм у повністю функціональну систему.

Програмна реалізація має повну зворотність криптоперетворень (операцій шифрування та розшифрування), що підтверджує внутрішню коректність й створює надійний фундамент для наступного етапу дослідницької роботи — проведення експериментального тестування статистичних та криптографічних характеристик шифру.

ПЕРЕЛІК ПОСИЛАНЬ

1. O'z DSt 1105:2009. Information technology. Cryptographic protection of information. Data encryption algorithm. — Tashkent: Agency "Uzstandard 2009. — 29 p. URL: https://csec.uz/ru/docs/OzDSt_1105.pdf

ДОДАТОК А ТЕКСТИ ПРОГРАМ

Програмна реалізація криптографічного алгоритму відповідно до державного стандарту Узбекистану O'z DSt 1105:2009.

А.1 Програма 1

```

1  """
2  O'z DSt 1105:2009 - Алгоритм шифрування даних (АШД)
3  Державний стандарт Узбекистану: симетричний блоковий шифр
4
5  =====
6  Параметри (§6.1)
7  =====
8
9  Блок          : 256 біт (32 байти)
10 Ключ k         : 256 або 512 біт
11 Функціональний kf : 256 біт
12 Кількість раундів : e = 8
13 Модуль         : p = 256
14
15 =====
16 Алгебраїчні примітки (доведено аналізом контрольного прикладу)
17 =====
18 Kse обчислюється як  $k + k' \cdot (1 + kf \cdot k)$  у цілочисельній арифметиці,
19 береться 672 старших бітів (§6.3.1.1).
20 Kst = kse[44:76] (ліві 256 біт правих 320 біт Kse, §6.3.1.2).
21 Діаграмація K будується за схемою §6.3.2.2:
22     діагональ = kss[6], d8 = kss[3], d9 = kss[4] тощо.
23 Ensure_invertible забезпечує непарність (непарність) всіх чотирьох
24 множників діагоналі (§5.1.4), що гарантує оборотність mod 256.
25 Aralash виконує  $\otimes_2$  (§5.1.5) створено mod 256.
26 Зворотна операція реалізована через  $16 \times 16$  лінійне обернення mod 256.
27 Масиви заміни BsA обчислюються через  $\text{row}(x, d, 257) \% 256$  (§6.3.1.4).
28 Epoch-ключі - ліві 256 бітів циклічно зсуненого Kse (§6.3.5).
29
30 """
31
32 from __future__ import annotations

```

```

31 from typing import List, Tuple, Optional
32
33 # =====
34 # Константи
35 # =====
36 BLOCK = 32          # байти (256 біт)
37 ROWS = 8            # рядки Holat
38 COLS = 4            # стовпці Holat
39 ROUNDS = 8         # раундів
40 P = 256             # арифметика mod p
41 KSE_B = 84          # Kse у байтах (672 біт)
42 KSE_N = 672         # Kse у бітах
43 STEP = 83           # зсув між ерсах-ключами (бітів)
44
45 H4 = List[List[int]] # матриця 4×4
46 H8 = List[List[int]] # масив 8×4
47
48
49 # =====
50 # Holat I/O
51 # =====
52
53 def _to_holat(data: bytes) -> H8:
54     """Завантаження 32-байт блоку у Holat[8][4] рядково (4 байти/рядок)."""
55     assert len(data) == BLOCK
56     return [[data[r * COLS + c] for c in range(COLS)] for r in range(ROWS)]
57
58
59 def _from_holat(h: H8) -> bytes:
60     """Вивантаження Holat[8][4] у 32 байти."""
61     return bytes(h[r][c] & 0xFF for r in range(ROWS) for c in range(COLS))
62
63
64 def _xor(a: H8, b: H8) -> H8:
65     """Qo'shHolat / Qo'shBosqichKalit: необхідний XOR (§6.3.6, §6.3.8)."""
66     return [[(a[r][c] ^ b[r][c]) & 0xFF for c in range(COLS)] for r in range(ROWS)]
67
68
69 # =====

```

```

70 # §6.3.1 Kse + масиви заміни
71 # =====
72
73 def _compute_kse(k: bytes, kf: bytes) -> bytes:
74     """
75     §6.3.1.1  $kse = k + k' \cdot (1 + kf \cdot k)$  у цілочисельній арифметиці.
76      $k'$  = праві 192 біти  $kf$ . Повертає ліві 672 біти.
77     """
78     k_i = int.from_bytes(k, 'big')
79     kf_i = int.from_bytes(kf, 'big')
80     kp_i = int.from_bytes(kf[8:], 'big') # останні 24 байти  $kf$ 
81     v = k_i + kp_i * (1 + kf_i * k_i)
82     bl = v.bit_length()
83     top = v >> (bl - KSE_N) if bl >= KSE_N else v << (KSE_N - bl)
84     return top.to_bytes(KSE_B, 'big')
85
86
87 def _bsa(d: int, R: int, L: int, b_swap: int) -> List[int]:
88     """§6.3.1.4 Масив заміни  $BsA[256]$  через  $\text{pow}(x, d, 257) \% 256$ ."""
89     arr = [pow(((i + L) % 256) + 1, d, 257) % 256 for i in range(256)]
90     b = b_swap
91     for i in range(1, 256):
92         if not ((i - arr[i]) % 256 != 0 and abs(arr[i - 1] - arr[i]) >= 8):
93             addr = (i - b) % 256
94             arr[i], arr[addr] = arr[addr], arr[i]
95             b = (b - 5) % 256
96     return arr
97
98
99 def _fix_d(d: int) -> int:
100     """§6.3.1.3 п.4-5: корекція парного степеня."""
101     if d % 2 == 0:
102         d = d - 1 if d % 4 == 0 else d + 1
103     if d % 2 == 1 and (d - 1) % 4 == 0:
104         d -= 2
105     return max(d, 3)
106
107
108 def shakl_seans_kalit_bayt(

```



```

109     k: bytes, kf: bytes
110 ) -> Tuple[List[int], List[int], List[int], List[int], bytes]:
111     """
112     §6.3.1 Обчислення Kse, Kst, масиву BSA.
113     Повертає (B1A, B2A, B1AD, B2AD, kst_bytes).
114     """
115     kse = _compute_kse(k, kf)
116
117     # §6.3.1.2: права 320-бітна частина Kse → Kst (32 б) + B (8 б)
118     kst = kse[KSE_B - 40 : KSE_B - 8]
119     B = list(kse[KSE_B - 8 :])
120
121     def param(j_val, j_zero=1):
122         return j_val if j_val else j_zero
123
124     d1 = _fix_d(B[0] if B[0] >= 3 else 3)
125     R1 = param(B[1]); L1 = param(B[2]); b3 = param(B[3])
126     d2 = _fix_d(B[4] if B[4] >= 3 else 3)
127     R2 = param(B[5]); L2 = param(B[6]); b7 = param(B[7])
128
129     B1A = _bsa(d1, R1, L1, b3)
130     B2A = _bsa(d2, R2, L2, b7)
131     B1AD = [0] * 256
132     for i, v in enumerate(B1A): B1AD[v & 0xFF] = i
133     B2AD = [0] * 256
134     for i, v in enumerate(B2A): B2AD[v & 0xFF] = i
135
136     return B1A, B2A, B1AD, B2AD, kst
137
138
139 # =====
140 # §6.3.2 Сеансовий ключ: діа матриці K1, K2
141 # =====
142
143 def _build_dm(kss: List[int]) -> H4:
144     """
145     §6.3.2.2 Діа матриця 4×4 зі спеціальною структурою.
146     Відображення kss → d: kss[6] = d7 (діагональ),
147     kss[3] = d8, kss[4] = d9, kss[5] = d3, kss[7] = d4,

```

```

148     kss[8] = d5, kss[9] = d6, kss[0..2] = d0..d2.
149
150     Мамуця:
151         d7   d0   d1   d2           kss[6]   kss[0]   kss[1]   kss[2]
152         d8   d7   d8   d8   =   kss[3]   kss[6]   kss[3]   kss[3]
153         d9   d3   d7   d9           kss[4]   kss[5]   kss[6]   kss[4]
154         d4   d5   d6   d7           kss[7]   kss[8]   kss[9]   kss[6]
155     """
156     return [
157         [kss[6], kss[0], kss[1], kss[2]],
158         [kss[3], kss[6], kss[3], kss[3]],
159         [kss[4], kss[5], kss[6], kss[4]],
160         [kss[7], kss[8], kss[9], kss[6]],
161     ]
162
163
164 def _ensure_inv(kss: List[int]) -> List[int]:
165     """
166     §6.3.2.1 Гарантує непарність всіх множників діагностичника mod 2,
167     що забезпечує оборотність  $\otimes_2$  mod 256.
168
169     Множники (для K1):
170         d7           = kss[6]
171         f1 = d7+d0+d8+d3+d5 = kss[6]+kss[0]+kss[3]+kss[5]+kss[8]
172         f2 = d7+d1+d8+d9+d6 = kss[6]+kss[1]+kss[3]+kss[4]+kss[9]
173         f3 = d7+d2+d8+d9+d4 = kss[6]+kss[2]+kss[3]+kss[4]+kss[7]
174     """
175     kss = kss[:]
176     for i in range(20):
177         if kss[i] == 0:
178             kss[i] = (kss[i] - 1) % P
179
180     # Діагональний елемент d7 має бути непарним
181     if kss[6] % 2 == 0: kss[6] = (kss[6] - 1) % P
182     if kss[16] % 2 == 0: kss[16] = (kss[16] - 1) % P
183
184     # Кожен множник fi має бути непарним
185     # K1 factors:           K2 factors (зсув +10):
186     for diag, others, target in [

```

```

187         (6, [0, 3, 5, 8], 8),      # f1 K1 → fix kss[8]
188         (16, [10,13,15,18], 18),   # f1 K2 → fix kss[18]
189         (6, [1, 3, 4, 9], 9),      # f2 K1 → fix kss[9]
190         (16, [11,13,14,19], 19),   # f2 K2 → fix kss[19]
191         (6, [2, 3, 4, 7], 7),      # f3 K1 → fix kss[7]
192         (16, [12,13,14,17], 17),   # f3 K2 → fix kss[17]
193     ]:
194         if (kss[diag] + sum(kss[o] for o in others)) % 2 == 0:
195             kss[target] = (kss[target] - 1) % P
196
197     return kss
198
199
200 def _build_linmap(K: H4) -> List[List[int]]:
201     """Буде 16×16 лінійне відображення для  $\otimes_2$  з фіксованим K mod 256."""
202     A = [[0] * 16 for _ in range(16)]
203     for i in range(16):
204         e = [[1 if r * 4 + c == i else 0 for c in range(4)] for r in range(4)]
205         res = _diamatrix(e, K)
206         for r in range(4):
207             for c in range(4):
208                 A[r * 4 + c][i] = res[r][c]
209     return A
210
211
212 def _invert_linmap(A: List[List[int]]) -> Optional[List[List[int]]]:
213     """Обернення 16×16 матриці mod 256 методом Гаусса-Жордана (зведена форма)."""
214     n = len(A)
215     aug = [A[i][:] + [1 if i == j else 0 for j in range(n)] for i in range(n)]
216     for col in range(n):
217         piv = next((r for r in range(col, n) if aug[r][col] % 2 == 1), None)
218         if piv is None:
219             return None    # не оборотна
220         aug[col], aug[piv] = aug[piv], aug[col]
221         inv_p = pow(int(aug[col][col]), -1, 256)
222         aug[col] = [(v * inv_p) % 256 for v in aug[col]]
223         for row in range(n):
224             if row != col and aug[row][col] % 256:
225                 f = aug[row][col]

```

```

226         aug[row] = [(aug[row][j] - f * aug[col][j]) % 256
227                     for j in range(2 * n)]
228     return [[aug[i][n + j] % 256 for j in range(n)] for i in range(n)]
229
230
231 def shakl_seans_kalit(kst: bytes) -> Tuple[H4, H4, List[List[int]], List[List[int]]:
232     """
233     §6.3.2 Буде K1, K2 та їх обернені лінійні відображення mod 256.
234     Повертає (K1, K2, A1_inv, A2_inv).
235     """
236     kss = _ensure_inv(list(kst[:20]))
237     K1 = _build_dm(kss[:10])
238     K2 = _build_dm(kss[10:20])
239     A1 = _build_linmap(K1)
240     A2 = _build_linmap(K2)
241     A1i = _invert_linmap(A1)
242     A2i = _invert_linmap(A2)
243     if A1i is None or A2i is None:
244         raise ValueError("Діаматриця не оборотна mod 256 - перевірте kst")
245     return K1, K2, A1i, A2i
246
247
248 # =====
249 # §5.1.5 / §6.3.3 Aralash: операція  $\otimes_2$  та її обернення
250 # =====
251
252 def _diamatrix(H: H4, K: H4) -> H4:
253     """
254     §5.1.5 Діаматричне множення  $H \otimes_2 K$  (mod 256).
255
256     Діагональні:
257          $h'[u, u] = h[u, u] \cdot \sum_i k[i, u] - \sum_{\{i \neq u\}} h[i, i] \cdot k[i, u]$ 
258
259     Позадіагональні:
260          $h'[s, u] = h[s, u] \cdot \sum_i k[i, u] + k[s, u] \cdot \sum_i h[i, u] - \sum_{\{i \neq s, u\}} h[s, i] \cdot k[i, u]$ 
261     """
262     n = 4
263     Hp = [[0] * n for _ in range(n)]
264     cs = [sum(K[i][u] for i in range(n)) % P for u in range(n)]
265     hcs = [sum(H[i][u] for i in range(n)) % P for u in range(n)]

```

```

265
266     for u in range(n):
267         Hp[u][u] = (H[u][u] * cs[u]
268                     - sum(H[i][i] * K[i][u] for i in range(n) if i != u)) % P
269
270     for s in range(n):
271         for u in range(n):
272             if s == u:
273                 continue
274             cross = sum(H[s][i] * K[i][u] for i in range(n) if i != s and i != u)
275             Hp[s][u] = (H[s][u] * cs[u] + K[s][u] * hcs[u] - cross) % P
276
277     return Hp
278
279
280 def _apply_inv(Ai: List[List[int]], H: H4) -> H4:
281     """Застосовує обернене лінійне відображення 16×16 до матриці 4×4."""
282     hf = [H[r][c] for r in range(4) for c in range(4)]
283     rf = [sum(Ai[i][j] * hf[j] for j in range(16)) % P for i in range(16)]
284     return [[rf[r * 4 + c] for c in range(4)] for r in range(4)]
285
286
287 def aralash(h: H8, K1: H4, K2: H4) -> H8:
288     """
289     §6.3.3 Шифрування: ліва половина Holat (рядки 0-3) ⊗2 K1,
290     права (рядки 4-7) ⊗2 K2.
291     """
292     return (_diatrix([h[r][:] for r in range(4)], K1)
293            + _diatrix([h[r][:] for r in range(4, 8)], K2))
294
295
296 def aralash_inv(h: H8, A1i: List[List[int]], A2i: List[List[int]]) -> H8:
297     """Розшифрування: застосовує обернені лінійні відображення до обох половин."""
298     return (_apply_inv(A1i, [h[r][:] for r in range(4)])
299            + _apply_inv(A2i, [h[r][:] for r in range(4, 8)]))
300
301
302 # =====
303 # §6.3.4 БайтАlmask: побайтова заміна

```

```

304 # =====
305
306 def bayt_almash(h: H8, Ba: List[int]) -> H8:
307     """§6.3.4 Заміна  $h[r][c] \rightarrow Ba[h[r][c]]$ ."""
308     return [[Ba[h[r][c]] & 0xFF for c in range(COLS)] for r in range(ROWS)]
309
310
311 # =====
312 # §6.3.7 Sur: циклічні зсуви
313 # =====
314
315 def sur_enc(h: H8) -> H8:
316     """
317     §6.3.7 (шифрування):
318     1. Стовець  $j \downarrow$  на  $(j+1) \bmod 8$  позиції.
319     2. Рядок  $i \rightarrow$  на  $(i+1) \bmod 4$  позиції.
320     """
321     h1 = [[0] * COLS for _ in range(ROWS)]
322     for col in range(COLS):
323         s = (col + 1) % ROWS
324         for row in range(ROWS):
325             h1[(row + s) % ROWS][col] = h[row][col]
326     h2 = [[0] * COLS for _ in range(ROWS)]
327     for row in range(ROWS):
328         s = (row + 1) % COLS
329         for col in range(COLS):
330             h2[row][(col + s) % COLS] = h1[row][col]
331     return h2
332
333
334 def sur_dec(h: H8) -> H8:
335     """
336     §6.3.7 (розшифрування):
337     1. Рядок  $i \leftarrow$  на  $(i+1) \bmod 4$  позиції.
338     2. Стовець  $j \uparrow$  на  $(j+1) \bmod 8$  позиції.
339     """
340     h1 = [[0] * COLS for _ in range(ROWS)]
341     for row in range(ROWS):
342         s = (row + 1) % COLS

```

```

343         for col in range(COLS):
344             h1[row][col] = h[row][(col + s) % COLS]
345     h2 = [[0] * COLS for _ in range(ROWS)]
346     for col in range(COLS):
347         s = (col + 1) % ROWS
348         for row in range(ROWS):
349             h2[row][col] = h1[(row + s) % ROWS][col]
350     return h2
351
352
353     # =====
354     # §6.3.5 Epoch-ключі
355     # =====
356
357 def _rot_left(v: int, s: int, b: int) -> int:
358     s %= b
359     m = (1 << b) - 1
360     return ((v << s) | (v >> (b - s))) & m
361
362
363 def _rot_right(v: int, s: int, b: int) -> int:
364     return _rot_left(v, b - s % b, b)
365
366
367 def build_epoch_keys(kse: bytes, encrypt: bool) -> List[H8]:
368     """
369     §6.3.5 Формує ROUNDS+1 масивів Ke[8][4].
370     Для розшифрування: початковий зсув + зворотні зсуви між раундами.
371     """
372     cur = int.from_bytes(kse, 'big') & ((1 << KSE_N) - 1)
373     if not encrypt:
374         init = (KSE_N - (ROUNDS * STEP) % KSE_N) % KSE_N
375         cur = _rot_left(cur, init, KSE_N)
376
377     keys: List[H8] = []
378     for step in range(ROUNDS + 1):
379         if step > 0:
380             cur = (_rot_left(cur, STEP, KSE_N) if encrypt
381                  else _rot_right(cur, STEP, KSE_N))

```

```

382         ke_int = (cur >> (KSE_N - 8 * BLOCK)) & ((1 << (8 * BLOCK)) - 1)
383         keys.append(_to_holat(ke_int.to_bytes(BLOCK, 'big')))
384     return keys
385
386
387 # =====
388 # §6.2 Шифрування / розшифрування блоку
389 # =====
390
391 def _enc_block(
392     pt: bytes,
393     K1: H4, K2: H4,
394     enc_keys: List[H8],
395     B1A: List[int], B2A: List[int],
396 ) -> bytes:
397     """
398     §6.2 Рис. 7 ECB-шифрування одного блоку.
399     e паундіє: XOR_key → Aralash → Sur → BaytAlmash
400     Фінал: XOR_key → Aralash
401     """
402     h = _to_holat(pt)
403     for step in range(ROUNDS):
404         h = _xor(h, enc_keys[step])
405         h = aralash(h, K1, K2)
406         h = sur_enc(h)
407         h = bayt_almash(h, B1A if (step + 1) % 2 == 1 else B2A)
408     h = _xor(h, enc_keys[ROUNDS])
409     h = aralash(h, K1, K2)
410     return _from_holat(h)
411
412
413 def _dec_block(
414     ct: bytes,
415     A1i: List[List[int]], A2i: List[List[int]],
416     enc_keys: List[H8],
417     B1AD: List[int], B2AD: List[int],
418 ) -> bytes:
419     """
420     §6.3 Рис. 7 ECB-розшифрування одного блоку.

```



```

421     Початок:  Aralash-1 → XOR_key[ROUNDS]
422     е паузів (зворотньо):  BaytAlmash-1 → Sur-1 → Aralash-1 → XOR_key
423     """
424     h = _to_holat(ct)
425     h = aralash_inv(h, A1i, A2i)
426     h = _xor(h, enc_keys[ROUNDS])
427     for step in range(ROUNDS, 0, -1):
428         h = bayt_almash(h, B1AD if step % 2 == 1 else B2AD)
429         h = sur_dec(h)
430         h = aralash_inv(h, A1i, A2i)
431         h = _xor(h, enc_keys[step - 1])
432     return _from_holat(h)
433
434
435     # =====
436     # Публічний API
437     # =====
438
439     class ASD:
440         """
441         О'з DSt 1105:2009 - Алгоритм шифрування даних.
442
443         Симетричний блоковий шифр: блок 256 біт.
444
445         Режими:
446         ECB (Elektron kod kitobi) - незалежні блоки
447         CBC (ShifrBlokIarni ilaktirish) - зчеплення блоків з IV
448
449         Приклад (ECB):
450         cipher = ASD(k_32b, kf_32b)
451         ct = cipher.encrypt_ecb(cipher.pad(plaintext))
452         pt = cipher.unpad(cipher.decrypt_ecb(ct))
453
454         Приклад (CBC):
455         ct = cipher.encrypt_cbc(cipher.pad(plaintext), iv_32b)
456         pt = cipher.unpad(cipher.decrypt_cbc(ct, iv_32b))
457
458         Параметри __init__:
459         k - 32 або 64 байти. Якщо 64: перші 32 = k, останні 32 = kf.

```

```

460     kf - 32 байти (необов'язково; якщо None - генерується з k).
461     """
462
463     def __init__(self, k: bytes, kf: Optional[bytes] = None):
464         if len(k) == 64:
465             kf, k = k[32:], k[:32]
466         elif len(k) != 32:
467             raise ValueError("k має бути 32 або 64 байти")
468         if kf is None:
469             kf = bytes((k[i] ^ k[(i + 7) % 32] ^ (i * 37 + 19)) & 0xFF
470                       for i in range(32))
471         if len(kf) != 32:
472             raise ValueError("kf має бути 32 байти")
473
474         self.k, self.kf = k, kf
475         self._init()
476
477     # === Ініціалізація =====
478
479     def _init(self) -> None:
480         B1A, B2A, B1AD, B2AD, kst = shakl_seans_kalit_bayt(self.k, self.kf)
481         self._B1A = B1A; self._B2A = B2A
482         self._B1AD = B1AD; self._B2AD = B2AD
483
484         self._K1, self._K2, self._A1i, self._A2i = shakl_seans_kalit(kst)
485
486         kse = _compute_kse(self.k, self.kf)
487         self._enc_keys = build_epoch_keys(kse, encrypt=True)
488         # Decrypt виходить в reverse order
489
490     # === ECB =====
491
492     def encrypt_ecb(self, data: bytes) -> bytes:
493         """ECB-шифрування. len(data) кратне 32."""
494         _chk(data)
495         return b"".join(
496             _enc_block(data[i:i+BLOCK], self._K1, self._K2,
497                       self._enc_keys, self._B1A, self._B2A)
498             for i in range(0, len(data), BLOCK)

```

```

499         )
500
501     def decrypt_ecb(self, data: bytes) -> bytes:
502         """ECB-розшифрування."""
503         _chk(data)
504         return b"".join(
505             _dec_block(data[i:i+BLOCK], self._A1i, self._A2i,
506                       self._enc_keys, self._B1AD, self._B2AD)
507             for i in range(0, len(data), BLOCK)
508         )
509
510     # === CBC =====
511
512     def encrypt_cbc(self, data: bytes, iv: bytes) -> bytes:
513         """CBC-шифрування."""
514         _chk(data); _chk_iv(iv)
515         out = bytearray()
516         prev = _to_holat(iv)
517         for i in range(0, len(data), BLOCK):
518             blk = _xor(_to_holat(data[i:i+BLOCK]), prev)
519             ctb = _enc_block(_from_holat(blk), self._K1, self._K2,
520                             self._enc_keys, self._B1A, self._B2A)
521             out += ctb
522             prev = _to_holat(ctb)
523         return bytes(out)
524
525     def decrypt_cbc(self, data: bytes, iv: bytes) -> bytes:
526         """CBC-розшифрування."""
527         _chk(data); _chk_iv(iv)
528         out = bytearray()
529         prev = iv
530         for i in range(0, len(data), BLOCK):
531             ct_b = data[i:i+BLOCK]
532             pt_b = _dec_block(ct_b, self._A1i, self._A2i,
533                               self._enc_keys, self._B1AD, self._B2AD)
534             pt_h = _xor(_to_holat(pt_b), _to_holat(prev))
535             out += _from_holat(pt_h)
536             prev = ct_b
537         return bytes(out)

```

```

538
539 # === Вирівнювання =====
540
541 def pad(self, data: bytes) -> bytes:
542     """Додає PKCS#7-сумісне вирівнювання до кратності 32."""
543     n = BLOCK - len(data) % BLOCK
544     return data + bytes([n] * n)
545
546 def unpad(self, data: bytes) -> bytes:
547     """Видаляє вирівнювання."""
548     if not data:
549         return data
550     n = data[-1]
551     if n == 0 or n > BLOCK:
552         raise ValueError("Некоректне вирівнювання")
553     return data[:-n]
554
555
556 # =====
557 # Внутрішні перевірки
558 # =====
559
560 def _chk(data: bytes) -> None:
561     if len(data) % BLOCK:
562         raise ValueError(f"Довжина {len(data)} не кратна {BLOCK}. "
563                          "Використайте cipher.pad(data).")
564
565 def _chk_iv(iv: bytes) -> None:
566     if len(iv) != BLOCK:
567         raise ValueError(f"IV має бути {BLOCK} байтів")
568
569 def demo() -> None:
570     print("=" * 68)
571     print(" 0'z DSt 1105:2009 - Алгоритм шифрування даних (АШД)")
572     print(" Симетричний блоковий шифр | блок 256 біт | e = 8 раундів")
573     print("=" * 68)
574
575     k = bytes.fromhex("37B60BBA0AB160CFDC18F50CDEE8E04530B3F8AF"
576                      "1432FE511FBB2029112F2143")

```

```

577 kf = bytes.fromhex("47E7694669C546B6FE163A89B0D896D6238B2315 "
578                      "32C404349CB0C7AA813DF96D ")
579 iv = bytes.fromhex("2654BB5FA375D89854EA489F9AA88416 "
580                      "FD4DEBBD9B3B403348F29FEE5234C37A ")
581
582 cipher = ASD(k, kf)
583 msg      = b"Confidential data protected by 0'z DSt 1105:2009!"
584 padded = cipher.pad(msg)
585
586 print(f"\nПовідомлення : {msg.decode()}")
587 print(f"Розмір (байт): {len(padded)}")
588
589 ct = cipher.encrypt_ecb(padded)
590 rt = cipher.unpad(cipher.decrypt_ecb(ct))
591 print(f"\n[ECB] CT : {ct.hex().upper()}")
592 print(f"[ECB] PT : {rt.decode()}")
593 print(f"[ECB]      : {'OK' if rt == msg else 'ПОМИЛКА'}")
594
595 ct2 = cipher.encrypt_cbc(padded, iv)
596 rt2 = cipher.unpad(cipher.decrypt_cbc(ct2, iv))
597 print(f"\n[CBC] CT : {ct2.hex().upper()}")
598 print(f"[CBC] PT : {rt2.decode()}")
599 print(f"[CBC]      : {'OK' if rt2 == msg else 'ПОМИЛКА'}")
600
601 if __name__ == "__main__":
602     demo()

```